

LabelEncoder#

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>

Description:

Encode target labels with value between 0 and n_classes-1. This transformer should be used to encode target values, i.e. y, and not the input X. Read more in the User Guide. Added in version 0.12.

Function Signature

```
class sklearn.preprocessing.LabelEncoder[source]#  
Attributes:  
classes_ndarray of shape (n_classes,)  
OrdinalEncoder  
OneHotEncoder  
class sklearn.preprocessing.LabelEncoder[source]#  
Attributes:  
classes_ndarray of shape (n_classes,)  
OrdinalEncoder  
OneHotEncoder  
fit(y)[source]#  
Parameters:  
yarray-like of shape (n_samples,)  
Returns:  
selfreturns an instance of self.
```

Parameters

yarray

Target values.

`yarray`

Target values.

`yarray`

Encoded labels.

`paramsdict`

Parameter names mapped to their values.

`yarray`

Target values.

`**paramsdict`

Estimator parameters.

`yarray`

Target values.

yarray

Labels as normalized encodings.

Code Examples

```
>>> from sklearn.preprocessing import LabelEncoder
>>> le = LabelEncoder()
>>> le.fit([1, 2, 2, 6])
LabelEncoder()
>>> le.classes_
array([1, 2, 6])
>>> le.transform([1, 1, 2, 6])
array([0, 0, 1, 2]...)
>>> le.inverse_transform([0, 0, 1, 2])
array([1, 1, 2, 6])
```

```
>>> le = LabelEncoder()
>>> le.fit(["paris", "paris", "tokyo", "amsterdam"])
LabelEncoder()
>>> list(le.classes_)
[np.str_('amsterdam'), np.str_('paris'), np.str_('tokyo')]
>>> le.transform(["tokyo", "tokyo", "paris"])
array([2, 2, 1]...)
>>> list(le.inverse_transform([2, 2, 1]))
[np.str_('tokyo'), np.str_('tokyo'), np.str_('paris')]
```

Notes & Warnings

NOTE

See also `OrdinalEncoder` to encode categorical features using an ordinal encoding scheme. `OneHotEncoder` to encode categorical features as a one-hot numeric array.

Detailed Documentation

Documentation

Encode target labels with value between 0 and `n_classes-1`.

This transformer should be used to encode target values, i.e. `y`, and not the input `X`.

Read more in the User Guide.

Added in version 0.12.

Holds the label for each class.

Encode categorical features using an ordinal encoding scheme.

Encode categorical features as a one-hot numeric array.

`LabelEncoder` can be used to normalize labels.

It can also be used to transform non-numerical labels (as long as they are hashable and comparable) to numerical labels.

Fitted label encoder.

Fit label encoder and return encoded labels.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A MetadataRequest encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Transform labels back to original encoding.

Set output container.

See [Introducing the set_output API](#) for an example on how to use the API.

Configure output of transform and fit_transform.

"default": Default output format of a transformer

"pandas": DataFrame output

"polars": Polars output

None: Transform configuration is unchanged

Added in version 1.4: "polars" option was added.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Estimator parameters.

Transform labels to normalized encoding.

Labels as normalized encodings.

© Copyright 2007 - 2025, scikit-learn developers (BSD License).